Contents

1 Explainable Interface Behavior Through Field Diagnostics **1**
1.1 Abstract . . . 2
1.2 1. Introduction . . . 2
1.2.1 1.1 The problem: behavior is a black box . . . 2
1.2.2 1.2 The diagnostic answer . . . 2
1.2.3 1.3 Contributions . . . 3
1.3 2. Background and related work . . . 3
1.4 3. The diagnostic taxonomy . . . 4
1.4.1 3.1 The modes, by question . . . 4
1.4.2 3.2 Three views worth drawing out . . . 4
1.4.3 3.3 The narrative reveal . . . 5
1.5 4. The reveal-never-mutate invariant . . . 5
1.5.1 4.1 The statement . . . 5
1.5.2 4.2 The two declared exceptions are not exceptions to the claim . . . 6
1.5.3 4.3 Where the code enforces it . . . 6
1.5.4 4.4 Why this makes the explanation trustworthy . . . 6
1.6 5. The runtime Inspector and lint self-audit . . . 7
1.6.1 5.1 The Platform Inspector reads the live registries . . . 7
1.6.2 5.2 `applyRecipe.inspect()` and real relationship resolution . . . 7
1.6.3 5.3 Lint surfaces quiet failures . . . 7
1.6.4 5.4 Passports: the static per-force explanation . . . 8
1.6.5 5.5 Together: a runtime that explains itself . . . 8
1.7 6. Evaluation plan . . . 8
1.8 7. Limitations . . . 9
1.9 8. Discussion . . . 9
1.10 9. Conclusion . . . 10
1.11 Appendix A. Reproducibility . . . 10
1.12 Appendix B. Conversion notes (markdown → preprint) . . . 11
1.13 Citation coverage . . . 11

1 Explainable Interface Behavior Through Field Diagnostics

Status: research draft (preprint, work in progress). Paper 8 of 8 in the Fundamental family — the diagnostics paper. Claims verified against the codebase and canonical docs as of 2026-06-26. See the series index and *the caveat canon* therein. This is a preprint draft, not canonical product documentation.

Post-verification note (0.8.1). Two diagnostic/authoring tools landed after the original draft and extend the self-explanation surface of §5: `mountXRay()` (`@fundamental-engine/dom`), a keystroke-toggled field-inspection overlay that surfaces the live runtime over any page, and `registerLintRule()` (`@fundamental-engine/dom`), which makes `lintPlatform()`'s rule set pluggable — a consumer can register a custom read-only guardrail rule alongside the built-in ones (§5.3). Both keep the reveal-never-mutate invariant of §4: they read runtime state and never write to the physics they report on.

Author: Zach Shallbetter **Series:** Fundamental Research Papers, Paper 8 of 8 **Companion paper (flagship):** Fundamental: A Field Translation Runtime for Relational DOM Interfaces. This paper assumes the vocabulary, the reciprocal model, and the truth-mode taxonomy established there — especially §8 ("the reverse half") and §6 (the force model). See the series index.

1.1 Abstract

An interface that *responds* to the user is the baseline. An interface that can *explain its own response* — name why an element moved, what is pulling attention, which relationship binds two regions, why the page calmed down — is rare, and on the web it is almost always reconstructed after the fact by a human reading source code. We present **field diagnostics**: the part of Fundamental that makes interface behavior inspectable. Fundamental computes interface behavior as a relational field (Paper 1); because that field is one shared, sampled state, the same state that *drives* a behavior can be *read back* to explain it. The contribution of this paper is threefold. First, a **diagnostic taxonomy**: a small family of read-only views — `force-vectors` (cause), `field-lines` (structure), `contours/potential` (terrain), `energy` (cost), `topology` (coupling), `causality` (why a thing moved), `prediction` (its expected path), and `inspector` (all of these at once) — each defined by *what state it reads from* and *what question it answers*. Second, the **reveal-never-mutate invariant**: every diagnostic visualizes field state without feeding back into the force computation it depicts. This is not a convention but a property the code is organized around and the design documents assert as canonical ("Diagnostic truth is read-only"); we argue it is what keeps the explanations *non-perturbing* — producing the view cannot change what it shows (no observer effect). Faithfulness of the labels is a separate matter, carried by the passport and conformance audits (§5). Third, the runtime's **self-explanation**: a live Platform Inspector that reads the running registries, an `applyRecipe.inspect()` surface that now reports real relationship resolution including unresolved endpoints, a `lintPlatform()` self-audit that surfaces quiet field failures, and per-force *passports* that statically declare what each behavior is. Together these let the runtime explain itself, live, without a debugger and without a second source of truth. We give a study *design* for whether diagnostic visibility improves author trust and time-to-diagnose; per the family's caveat canon, no results are claimed.

1.2 1. Introduction

1.2.1 1.1 The problem: behavior is a black box

Web interface behavior is usually opaque to the person who authored it. When an element grows heavier, brightens, slides, or settles, the author cannot ask the interface *why*. The cause is distributed across CSS rules, transition timings, event handlers, and framework state, and the only way to recover it is to read the code that produced it and simulate it in one's head. Animation debuggers and devtools help, but they explain *what the browser is doing* (which transition is running, which style won the cascade), not *why the interface decided to do it*. The gap is between mechanism and reason.

Fundamental makes this gap worse before it makes it better. A relational field (Paper 1) is *more* expressive than local component state — emphasis transfers across the page, a saturated region lights its neighbours, a worn reading trail biases future attention — and a richer behavior is a richer thing to be confused by. The motivating questions of the whole family are diagnostic in form: *"why is this element emphasized? what is pulling attention? which relationship is active? what caused this state?"* (Paper 1, §1.1). A field runtime that could not answer them would have traded one black box for a larger one.

1.2.2 1.2 The diagnostic answer

Fundamental's answer is that the field is not only a *mechanism* but an *explanation surface*. Because the runtime computes behavior as one shared, sampled field state (Paper 1, §3), the very state that moves matter and writes back to the DOM can be *read again* and rendered as an explanation. The system already holds the cause; a diagnostic simply makes it visible. The canonical framing is blunt:

> Visualization is not decoration. It is how the system explains itself. *(visualization taxonomy, "Purpose")*

This paper isolates the diagnostic framework as its own contribution. It is not the paradigm (Paper 1), not the runtime architecture (Paper 5), not patterns (Paper 6), data (Paper 7), accessibility (Paper 4), or the reading/evidence studies (Papers 2–3); it cross-references each. Its single claim is:

Interfaces should not only respond — they should be able to explain their response. Fundamental's diagnostic framework makes interface behavior inspectable through causality, topology, prediction, potential, energy, and relationship diagnostics, without those diagnostics ever mutating the behavior they reveal.

1.2.3 1.3 Contributions

1. **A diagnostic taxonomy** (§3). A small, closed family of read-only views, each defined by *what field state it reads from* and *what question it answers* — mapping the modes (`force-vectors`, `field-lines`, `contours/potential`, `energy`, `topology`, `causality`, `prediction`, `inspector`) onto the questions a field runtime should be able to answer.
2. **The reveal-never-mutate invariant as an auditable property** (§4). Diagnostics visualize field state and never feed back into `apply()`. Particles are the only "visualization" that is itself state; everything else is strictly read-only. We ground this in the visualization truth table and the behavior table's "Diagnostic truth is read-only," and argue it is the source of the explanations' trustworthiness.
3. **The Inspector and lint as the runtime's self-explanation** (§5). The Platform Inspector reads the live registries; `applyRecipe.inspect()` reports per-element metrics and *real* relationship resolution including unresolved endpoints; `lintPlatform()` surfaces quiet failures; per-force passports declare each behavior statically. The runtime can explain itself, live.

1.3 2. Background and related work

Explainability and provenance in information visualization. A long line of InfoVis work treats a visualization's job as not only display but *accountability* — showing where a value came from, how confident it is, and what would change it. Provenance and uncertainty visualization, and the broader "explainable visualization" agenda, motivate the stance that a view should expose its own basis. Fundamental adopts this at the level of interface *behavior* rather than data: the thing being made accountable is the interface's own response. [ragan2016; hullman2019]

Debugging and inspector tooling. Browser devtools, the DOM/style inspectors, animation inspectors (timeline scrubbing, easing editors), and layout debuggers (flexbox/grid overlays, paint-flashing) are the closest practical relatives. They explain *what the engine did*. Fundamental's Inspector explains *why the field decided* — it reads the runtime's own registries and per-force contributions rather than the browser's rendering pipeline. The lineage is the same (a live, read-only window into a running system) but the explanandum differs. [chrome-devtools; firefox-devtools]

"Why" and causal explanation in user interfaces. Work on explanation in intelligent and context-aware UIs (the "Why did the system do that?" question, scrutable and intelligible context-aware systems, explanation interfaces for recommenders) frames the user-facing form of the problem. Fundamental contributes a *mechanical* answer for a deterministic interface runtime: causality is decomposed into per-force contributions, not inferred. [lim2009; kulesza2013; abdul2018]

Force-directed and physical-metaphor systems. Force-directed layout and physics-based interaction toolkits render forces and trajectories for debugging, but typically as a development aid distinct from the production behavior. Fundamental's distinguishing stance is that the *same* field state drives the behavior and its explanation, and that the read-only boundary between them is an enforced invariant rather than a coding convention (§4). [eades1984; fruchterman1991; bostock2011]

The external citations above are resolved against the family bibliography (see `references.md`); none are fabricated.

1.4 3. The diagnostic taxonomy

A diagnostic is a *view*: it takes some part of the field state and renders it so a person can read a property that is otherwise invisible. Fundamental's diagnostics form a small, closed family. Each is defined by two things — *what it reads from* and *what question it answers* — and the discipline of the taxonomy is that those two are never confused (the lane hygiene of Paper 1, §6.6: diagnostics are a lane distinct from forces and metrics; `potential` is a diagnostic, never a token).

The canonical mnemonic states the family compactly:

```
Particles show matter.        Field lines show structure.
Force vectors show cause.     Trails show history.
Heatmaps show accumulation.   Contours show terrain.
Energy views show cost.       Topology shows relationships.
DOM state shows reciprocity.
```

(visualization taxonomy, "Purpose")

1.4.1 3.1 The modes, by question

Diagnostic mode	Reads from	Answers the question
`force-vectors`	a force's `apply()` Δv on a probe at a point	*what is pushing here, and how hard?*
`field-lines`	a force's `field()` structure	*what shape does this body radiate?*
`streamlines`	the net vector field	*which way would matter flow?*
`contours`	a scalar field's iso-values	*where are the equal-value bands?*
`potential`	the potential field `Φ`	*where are the wells and gradients?*
`energy`	particle + field state	*what does this behavior cost; is it drifting?*
`topology`	relationship agents + positions	*which elements are coupled, how strongly?*
`causality`	per-token `apply()` contributions	*why did this element move?*
`prediction`	a deterministic forward ghost step	*where is this matter expected to go?*
`inspector`	the whole system snapshot	*all of the above, live, at once*

This table is not aspirational: `VISUALIZATION_TRUTH_TABLE`, the `RENDER_MODES` catalog, and `VISUALIZATION_PRESETS` ship as data in `packages/core/src/visual/visualization.ts`, and every catalog mode is marked `shipped`. The diagnostic math behind the overlays ships in `packages/core/src/diagnostics/` and the canvas drawing of them in `diagnostics/render.ts` (the C1 overlays: `force-vectors`, `contours`, `potential`, `energy`) and `diagnostics/modes.ts` (`topology`, `inspector`, `causality`, `prediction`). The diagnostic math is pure and unit-tested (`diagnostics.test.ts`); the `/docs/diagnostics` page renders the overlays over *demo* data, while the Platform Inspector (§5.1) reads *live* runtime state.

1.4.2 3.2 Three views worth drawing out

Causality — "why did that move?" The behavior table and the visualization taxonomy both pose this as the question a field runtime must answer. The implementation decomposes the motion at a point into per-token contributions. `causalityAt(registry, tokens, body, x, y, probe)` (`packages/core/src/diagnostics/probes.ts`) fires a probe through each force's `apply()` and records the Δv each token contributes; `causalityBars()` (`diagnostics/modes.ts`) ranks those contributions by magnitude and reports each token's *fraction* of the total. The result is a literal answer to *why*: a ranked list — "this matter moved 0.61 because of `attract`, 0.24 because of `swirl`, 0.15 because of `magnetism`" — and a vector per contributing force drawn from the probe. Crucially this works *because* Fundamental separates structure from cause (Paper 1, §3.3): there is a real `apply()` to attribute the motion to, and a real probe to read it with — *field magnitude is not force magnitude*, so a still or neutral probe correctly reads zero from a velocity- or charge-dependent force.

Prediction — "where will it go?" `ghostTrajectory()` (`diagnostics/modes.ts`) integrates one probe forward under the body forces — summing each (force, body) Δv per step and damping by the same friction

the live integrator uses — and returns the path *without touching live state*. Its docstring states the property the mode depends on: *"Pure and repeatable: same inputs → same path. This is the prediction overlay's data (an expected future path), not the live sim."* The expected path is shown as a fading dashed ghost; the divergence between the ghost and the actual trail is itself a diagnostic (integrator stability, force comparison, teaching).

Topology — "what is connected?" `topologyEdges(agents, posOf)` resolves relationship agents to drawable edges, *dropping any whose endpoints cannot be placed* ("you can only draw what you can place"), and `drawTopology()` renders coupling as line width (strength), glow (accumulated memory), and accent (active this tick). The relationship lane is the platform's typed graph (Paper 1, §5.2); the topology diagnostic is its read-only rendering.

1.4.3 3.3 The narrative reveal

The diagnostics are also composable into a *teaching* order — a progressive reveal that names each layer as it appears: particles, then bodies, then field lines, then forces moving agents, then density writing to the DOM, then heatmaps, then topology, then the inspector showing reciprocity (visualization taxonomy §13). This narrative ships, and — relevant to Paper 4 — collapses under `prefers-reduced-motion` to a static preview that still names each layer and its current state, so the *explanation* reaches keyboard and reduced-motion users without depending on animation. The accessible form of an explanation is itself part of the diagnostic contract; the conformance model for it is Paper 4's.

1.5 4. The reveal-never-mutate invariant

The taxonomy's value rests on one property: a diagnostic reveals field state and **must not mutate the physics it depicts unless mutation is explicitly declared as feedback** (Paper 1, §8). This section states the invariant, shows where the code and canon enforce it, and argues it is the source of the explanations' trustworthiness.

1.5.1 4.1 The statement

The canonical behavior table is unambiguous:

> **Diagnostic truth is read-only.** All diagnostic render modes — including `causality` and `prediction` — only *visualize* field state. They read the field and draw on a render surface; they never feed back into `apply()` or mutate physics. *(fundamental field-behavior table)*

The visualization truth table makes the boundary precise, column by column. It records, for every mode, *what it reads from* and *whether it mutates physics*:

Visualization	Reads from	Mutates physics?
Particles	particle state	**yes — particles *are* state**
Field lines	`field()`	no
Force vectors	`apply()` or probe	no
Contours / Potential	scalar / potential field	no
Energy	particle + field state	no
Topology	relationship agents	optional
DOM state	CSS variables and events	**yes, visibly — this is feedback**
Causality	per-force contributions	no
Prediction	deterministic ghost step	no

(visualization taxonomy, "Visualization Truth Table"; abridged)

Two rows are the boundary cases that make the rule meaningful, and they are exactly the two the next subsection isolates.

1.5.2 4.2 The two declared exceptions are not exceptions to the claim

Particles. Particles are the one "visualization" that is *itself* state — they are matter, and matter is part of the field. Drawing a particle does not mutate physics; *being* a particle does. So the "yes" in the particle row is not a diagnostic feeding back into behavior; it is the observation that particles were never a diagnostic in the first place. All the actual diagnostics — the ones that answer *why*, *where*, *what cost*, *what coupling* — are read-only.

DOM state / feedback. The DOM-state row is "yes, visibly" because the reverse half of the reciprocal loop (Paper 1, §8) *does* mutate visible DOM — that is the point of the system. But this mutation is **declared feedback**, not a diagnostic side effect: it routes only through the `FeedbackRegistry`, writes only CSS custom properties (`--field-*`), and is opt-in per body (`data-feedback`). The boundary is therefore not "nothing ever changes the DOM" but "the only thing that changes behavior is *declared* behavior; diagnostics are not." A diagnostic reading `--field-density` to draw a bar (as the Inspector's reciprocity panel does, §5.3) reads the feedback; it does not produce it.

1.5.3 4.3 Where the code enforces it

The invariant is structural, not aspirational. The diagnostics module's own index says so — *"Nothing mutates physics"* — and each diagnostic is written as a pure data/geometry function plus a thin `draw*` helper:

- `energyReport()` and `energyDrift()` (`diagnostics/energy.ts`) are pure folds over a particle array; the docstring notes *"it reads state, never mutates it (the energy view must not affect integration)."*
- `forceVectorAt()` (`diagnostics/probes.ts`) constructs a throwaway probe particle, calls `apply()` on *it*, and returns `p.vx - probe.vx` — the Δv on the instrument, never on any live particle. It even catches and zeroes any exception so an instrument can never perturb a frame.
- `ghostTrajectory()` integrates a *local* probe in local variables; the live integrator is untouched.
- `topologyEdges()`, `causalityBars()`, and `inspectorRows()` are pure transforms from state to drawable rows.

The render helpers take a `CanvasRenderingContext2D` and draw; they are given no handle to the registries' write paths. This mirrors the platform's own self-audit posture: `lintPlatform()` likewise *"reads — it never mutates state, physics, or the DOM"* (Paper 1, §5.3). The diagnostic layer and the lint layer are the same idea applied to two surfaces.

1.5.4 4.4 Why this makes the explanation trustworthy

An explanation is only worth trusting if producing it cannot change the thing being explained. A debugger that perturbs the program it inspects (a Heisenbug) is a familiar hazard; a behavior explanation that subtly nudged the behavior would be worse, because the user would be shown a reason that the act of showing had altered. The reveal-never-mutate invariant forecloses *that* failure: a Fundamental diagnostic cannot **perturb** the behavior it reports, because it has no channel through which to change what it shows. The same field state drives both the behavior and the picture of it, and the picture has no write access. (This is non-perturbation, not fidelity: a read-only view can still mislabel or mis-aggregate — that the labels are *faithful* rests on the passport and conformance audits of §5.) We regard this as the diagnostic analogue of the family's broader epistemic stance — truth modes, passports, conformance, lint (Paper 1, §6, §9.2) — where honesty is *mechanical* rather than rhetorical: the diagnostic's read-only-ness is a property the code is shaped by, not a promise in prose.

1.6 5. The runtime Inspector and lint self-audit

Diagnostics (§3) answer point questions about matter and forces. The **Inspector** answers the system-level question — *is this field actually measured, related, and fed back, or is it magic?* — by reading the runtime's own self-description, live. It is the "all of the above" mode of the taxonomy, realized as a running surface.

1.6.1 5.1 The Platform Inspector reads the live registries

The Platform Inspector shipped in #198 (`feat(inspector): live Platform Inspector reading the running field-root`). It reads the running `<field-root>`'s platform — the production Phase-D runtime, not a mock — each frame: the six-phase scheduler's `frame` counter and phase order, and the live counts and contents of the registries (`measure.size`, `state.elements()`, `feedback.boundVars()`, `relationships.all()`, `visuals.all()`, `overlays.all()`). The page at `apps/site/src/pages/docs/inspector.astro` states the intent precisely: it answers *"why is this moving, what's influencing it, which relationship binds these?"*.

Two design choices keep the Inspector honest about *itself*:

- **It reads on-phase only.** The live readout uses `measure.last()`, never `measure()`, so observing the runtime never triggers an off-phase layout read — the Inspector obeys the same scheduler discipline it reports on (Paper 1, §5.1). It is a read-only observer of a read/write-disciplined loop.
- **Its zeroes are real.** When the docs page declares no visual bindings or overlays, the Inspector shows zero for them. The page says so explicitly: *"these are live, not mocked — and the zeroes are real too."* An inspector that fabricated activity would defeat its own purpose.

1.6.2 5.2 `applyRecipe.inspect()` and real relationship resolution

A pattern (Paper 6) applied to live content exposes `inspect()` (`packages/dom/src/apply-recipe.ts`), returning per-element metrics plus relationship counts. The load-bearing recent change is #222 (`fix(platform): compute real relationship resolution - stop hardcoding relResolved = relTotal`). Before it, resolution was assumed; after it, the runtime *resolves* each declared relationship against the DOM and distinguishes:

- `relationships` — resolved edges (both endpoints known);
- `relationshipsUnresolved` — declared edges whose target id-ref resolves to **no element**;
- `relationshipResolution` — `resolved / (resolved + unresolved)`;
- `unresolvedRelationships` — the actual `{ from, type, target }` of each broken endpoint.

This makes the inspector able to *name a citation that points at nothing.* The relationship registry (`relationships.ts`) tracks unresolved declarations rather than silently dropping them (`unresolvedAll()`), so a body that cites `#source-7` when no `#source-7` exists shows up as one unresolved edge — and, in the metric computation, lowers that body's relationship resolution and raises its entropy rather than being invisibly ignored. The diagnostic value is that *absence is reported as absence*: an explanation that quietly omitted the broken link would be a worse explanation than one that named it. (The same fix is what lets Paper 7's data-bound studies report honest resolution.)

1.6.3 5.3 Lint surfaces quiet failures

`lintPlatform()` (`packages/dom/src/lint.ts`) is the platform's self-audit. A field layer *fails quietly*: a relation points at a missing id, a visual duplicates text the screen reader already reads, an element is styled from state it was never registered for. None of these is a type error; all are data pathologies. The lint rules surface them as warnings:

```
relation-target-missing   a [data-field-relation] points at a missing/unregistered body
state-unregistered        an element holds field state but was never registered to measure
overlay-without-links     a relationship overlay has no relationships behind it
feedback-non-css-var      a binding writes ARIA/attributes instead of --field-* (an error)
```

`measurement-off-phase` layout was read outside the read phase
`visual-orphan / -not-hidden` a visual has no semantic source, or a decorative layer is not aria-hidden

Each rule is a pure function over the registries (testable without a live page), and `lintPlatform()` aggregates them; the Inspector renders the live result, severity-colored. Lint is to the *platform* what the passport validator is to *forces* and conformance is to *physics*: a continuously checkable audit. And like every diagnostic, *lint reads; it never mutates state, physics, or the DOM* (Paper 1, §5.3).

1.6.4 5.4 Passports: the static per-force explanation

Where the Inspector explains the *live* system, a **passport** (`packages/core/src/contracts/passport.ts`) is the *static* per-force explanation — the answer to "what is this behavior?" independent of any frame. Each passport declares, machine-readably: family and class; truth mode; whether the force owns a `field()`; whether it reads `env.fieldAt()`; whether it moves particles, does work, conserves speed, requires charge or velocity, affects neutral matter; and the render modes it reads best in. Derived facts (`canVisualizeFieldLines = ownsField`, `canVisualizeForceVectors = movesParticles`) are *computed* from the structural ones in `finish()`, so a passport cannot contradict itself, and `validatePassports()` cross-checks every passport against the live registry and the conformance catalog, so it cannot drift from the force it describes. The Inspector renders all of them as a table, and the system report (`systemReport()`) asserts that *every* force is passported and conformance-covered. The passport is why a diagnostic can label a contribution `magnetism` and have that label *mean* something auditable — "perpendicular to velocity, does no work, requires charge" — rather than being a free-text guess.

1.6.5 5.5 Together: a runtime that explains itself

The pieces compose into one claim: the runtime can explain itself, live, from its own state. The passports say what each force *is*; the diagnostics say what the forces are *doing* (cause, terrain, cost, coupling, prediction); the Inspector says what the live system *holds* (measurements, state, feedback, relationships); `inspect()` says how well the declared structure *resolved* (including what did not); and lint says where the field is *quietly broken*. No second source of truth, no separate instrumentation build, no human reconstruction from source — and, per §4, no channel by which any of it can alter the behavior it reports.

1.7 6. Evaluation plan

Per the family's caveat canon (item 6), **no user-study results exist yet**; what follows is a *design*, framed as a plan, with no outcomes claimed. The hypothesis is that diagnostic visibility improves **author trust** and **debuggability** of field behavior.

Question. When a Fundamental interface behaves unexpectedly, do the diagnostics (causality, prediction, topology, the Inspector, lint) let an author *correctly* identify *why* — faster, and with fewer false assumptions — than without them?

Design. A within-subjects task study. Authors are given a Fundamental page exhibiting a seeded behavior ("this heading is unexpectedly heavy"; "these two sections are coupled but should not be"; "a citation is not lighting its source") and asked to diagnose the cause. The **diagnostics-on** condition exposes the Inspector, the causality/topology overlays, and lint; the **diagnostics-off** condition exposes only the rendered page and the source. Tasks and conditions are counterbalanced.

Measures (plan).

- *Time-to-diagnose* — wall-clock from task start to a stated, correct cause.
- *Correct identification of why* — whether the author names the actual responsible force / unresolved relationship / off-phase read, scored against ground truth (which the runtime itself knows: the causality ranking, the `unresolvedRelationships` list, the lint warning).
- *False-assumption rate* — incorrect causal claims stated with confidence before reaching the right one.

- *Author-trust rating* — a post-task subjective measure of how much the author trusts the interface's behavior, with vs without diagnostics.

Ground truth is available without a human oracle. Unusually for a debuggability study, the *correct* answer is computable from the runtime: the seeded cause corresponds to a specific causality contribution, a specific entry in `unresolvedRelationships`, or a specific lint code. This lets correctness be scored mechanically and lets seeded behaviors be generated rather than hand-authored.

What it cannot show. The study measures author-facing debuggability, not end-user comprehension (§7). It also cannot, on its own, validate the *honesty* of the explanations — that is the job of the passport/conformance/lint audits, which run continuously in CI independent of any study.

No numbers are reported here; this is a protocol, consistent with how Papers 2–3 frame their studies.

1.8 7. Limitations

1. **Diagnostics explain the field's computation, not the human-perceived meaning.** Causality can say *attract contributed 0.61 of this motion*; it cannot say *and that is why the reader felt the heading mattered*. The diagnostic answers the mechanical "why," not the phenomenological one. Whether the field's computed emphasis matches felt importance is an empirical question for Papers 2–3, not a diagnostic guarantee.
2. **An inspector helps experts more than end users.** The Inspector, passport table, and causality bars are authoring instruments. They make the *author's* model of the interface inspectable; they are not an end-user feature, and we do not claim end users read them. (The reduced-motion narrative reveal, §3.3, is the closest thing to an end-user-facing explanation, and even it is primarily pedagogical.)
3. **The explanations are only as honest as the truth-mode classification.** A diagnostic labels a contribution by its passport; if a passport mis-declared a force, the explanation would inherit the error. This risk is real but *bounded by audit*: `validatePassports()` cross-checks every passport against the live registry and conformance catalog, and `systemReport()` asserts full coverage (Papers 1 and 5). The honesty is mechanical, but it is honesty *about the classification*, which is itself only as good as the conformance experiments behind it.
4. **Some diagnostics are visual and need a screen.** Field lines, force vectors, contours, and the ghost trajectory are inherently spatial; they assume a render surface and sighted reading. The accessible substitutes — the named-layer narrative, the textual Inspector rows, the lint message list, the `unresolvedRelationships` names — carry the *propositional* content (which force, which broken link, which warning) but not the *spatial gestalt*. The reduced-motion / non-visual equivalence model is Paper 4's, and we defer the conformance argument there.
5. **Causality and prediction are exact for class-A (body→particle) forces.** The probe instruments read body→particle forces directly; class-B/C/D forces (pairwise, grid, shape) need neighbours or grids that the conformance runner wires, so their per-token attribution is less direct. The taxonomy ships for all modes, but causal *decomposition* is sharpest where a single probe suffices.

1.9 8. Discussion

Why explainability belongs *in* the runtime, not bolted on. The conventional shape of interface explainability is a separate instrument: a debugger, a logger, an inspector that *observes* a system it is not part of and *reconstructs* what the system must have done. Reconstruction can drift — the instrument's model of the behavior and the behavior itself are two artifacts that must be kept in sync, and they routinely fall out of sync. Fundamental's diagnostics are not a second artifact. They read the *same* field state that produces the behavior. There is nothing to keep in sync because there is only one source: the explanation and the behavior are two reads of one state. This is the structural reason the explanation *cannot* drift from the behavior — and it is only available because the runtime computes behavior as inspectable field state in the first place (Paper 1).

The read-only boundary is what turns "inspectable" into "trustworthy." It is not enough that the state is readable; it must be readable *without the act of reading changing it.* §4's invariant — the diagnostic has no write channel to the physics — is what lets an author believe the picture. Combined with the static honesty of passports and the continuous audit of lint and conformance, Fundamental's claim is not merely "you can see what the field is doing" but "producing the view cannot have changed the answer, and the labels you see are audited — by passports and conformance — rather than merely asserted."

Cost. Diagnostics are not free; rendering overlays and running lint every frame has a price, and on a simple, flat interface that price may exceed the explanatory benefit — the same "does the field pay for itself" threat the flagship raises (Paper 1, §10). The mitigation is that diagnostics are opt-in views over state the runtime already computes; the *explanation* costs extra only when an author asks for it. The state was always there.

1.10 9. Conclusion

Interfaces should be able to explain their responses, and on Fundamental they can, because behavior is computed as one shared, sampled field that the runtime can read back as an explanation. We presented the diagnostic framework as three contributions: a closed taxonomy of read-only views, each defined by what it reads and what it answers (`force-vectors`, `field-lines`, `contours/potential`, `energy`, `topology`, `causality`, `prediction`, `inspector`); the reveal-never-mutate invariant, grounded in the visualization truth table and the behavior table's "Diagnostic truth is read-only," which makes the explanations trustworthy by denying them any channel to change what they show; and the runtime's self-explanation — a live Platform Inspector over the registries, an `inspect()` surface that reports *real* relationship resolution including unresolved endpoints, a `lintPlatform()` self-audit for quiet failures, and per-force passports as the static account of each behavior. We gave an evaluation *design* for author trust and debuggability and claimed no results, per the caveat canon, and we were explicit about what diagnostics do not explain: human-perceived meaning, end-user comprehension, and anything the classification beneath them gets wrong. The diagnostic framework is Fundamental's answer to its own founding questions — *why did this move? why is this emphasized? why are these connected? why did the page calm down?* — answered not by a human reading the source, but by the runtime, from its own state, live, and read-only.

1.11 Appendix A. Reproducibility

Every diagnostic claim in this paper is checkable against the repository.

- **The diagnostics module:** `packages/core/src/diagnostics/` — `energy.ts` (energy accounting), `potential.ts` (scalar `potentialAt` + grid sampling for contours/potential), `probes.ts` (`forceVectorAt`, `causalityAt`, the probe presets), `fields.ts` (heatmap-variant samplers), `modes.ts` (`topologyEdges`/`drawTopology`, `inspectorRows`/`drawInspector`, `causalityBars`/`drawCausality`, `ghostTrajectory`/`drawPrediction`), `render.ts` (the C1 canvas overlays), and `index.ts` (the `DIAGNOSTICS` list, *"Nothing mutates physics"*).
- **The render-mode catalog and truth table:** `packages/core/src/visual/visualization.ts` (`VISUALIZATION_TRUTH_TABLE`, `RENDER_MODES`, `VISUALIZATION_PRESETS`).
- **The Platform Inspector:** `apps/site/src/pages/docs/inspector.astro` (live registry readout, pattern inspector, reciprocity panel; shipped #198, pattern inspector #211) and the live `apps/site/src/pages/docs/diagnostics.astro` page.
- **The platform self-audit:** `packages/dom/src/lint.ts` (`lintPlatform` and the per-rule pure functions).
- **Relationship resolution including unresolved endpoints:** `packages/dom/src/apply-recipe.ts` (`inspect()` → `AppliedRecipeInspection`) and `packages/dom/src/relationships.ts` (`unresolvedAll`, `scanRelationships`); the real-resolution fix is #222.
- **Per-force passports:** `packages/core/src/contracts/passport.ts` (`ForcePassport`, `PASSPORTS`, `validatePassports`) and the system report `packages/core/src/inspect/report.ts` (`systemReport`).

The canonical design documents corroborate the framing: `docs/canonical/visualization-methods-taxonomy.md` (the diagnostic taxonomy and truth table) and `docs/canonical/fundamental-field-behavior-table.md` ("Diagnostic truth is read-only").

1.12 Appendix B. Conversion notes (markdown → preprint)

Notation is kept LaTeX-compatible (inline math and fenced blocks translate directly). Figures referenced in prose but not yet drawn — the diagnostic-taxonomy map (§3.1), a causality bar/vector decomposition (§3.2), an actual-vs-ghost trajectory with divergence (§3.2), and the Inspector registry-readout layout (§5.1) — are produced at conversion time from these descriptions. External citations must be resolved against `references.md` and verified before submission; none are fabricated.

1.13 Citation coverage

- InfoVis explainability / provenance / uncertainty visualization (§2) — the basis for "a view should expose its own basis": [ragan2016; hullman2019].
- Browser devtools, animation inspectors, and layout debuggers (§2) — the closest practical relatives, for the "what the engine did vs why the field decided" contrast: [chrome-devtools; firefox-devtools].
- Intelligibility and "Why did the system do that?" explanation in intelligent / context-aware UIs (§2) — the user-facing form of causal explanation: [lim2009; kulesza2013; abdul2018].
- Force-directed / physics-based interaction debugging tools (§2) — for the "same state drives behavior and explanation" distinction: [eades1984; fruchterman1991; bostock2011].
- Heisenbug / observer-effect wording is treated as an analogy to non-perturbing diagnostics, not as a separately claimed empirical result.